UNIVERSIDADE FEDERAL DE RORAIMA

DCC403 – SISTEMAS OPERACIONAIS I

PROGRAMAÇÃO CONCORRENTE COM THREADS

PIPELINE DE PROCESSAMENTO (MULTISTAGE)

SARAH CAROLINE AMARAL PEREIRA

OBJETIVO DO TRABALHO

DESENVOLVER UM SISTEMA CONCORRENTE UTILIZANDO THREADS, IMPLEMENTANDO UM **PIPELINE** COM **TRÊS ESTÁGIOS**:

- LEITURA;
- PROCESSAMENTO;
- ESCRITA;

E EXPLORAR A COMUNICAÇÃO ENTRE AS THREADS, A SINCRONIZAÇÃO E O PARALELISMO.

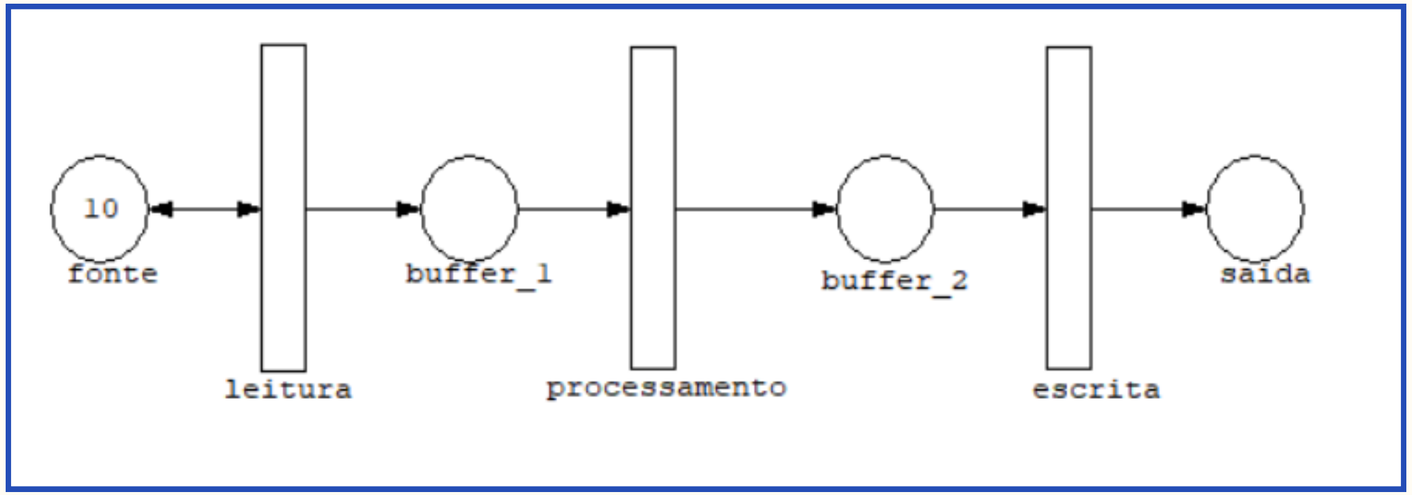
PROBLEMA

No modelo sequencial:

leitura > processamento > escrita

Cada etapa só começa quando a anterior termina, e isso nos causa problemas de baixo aproveitamento de tempo e recursos. E o objetivo deste trabalho é permitir que essas etapas executem simultaneamente.

MODELAGEM COM REDE DE PETRI



Antes da implementação, o sistema foi modelado com Rede de Petri, para representar visualmente o fluxo de dados e o comportamento concorrente.
E permite analisar dependências, sincronização e o fluxo entre etapas.

ESTRUTURA DO PIPELINE E COMUNICAÇÃO

O sistema foi dividido em três estágios:

leitura > processamento > escrita

Cada estágio é executado por uma thread. E a comunicação entre esses estágios é feita por buffers (filas):

leitura > buffer 1 > processamento > buffer 2 > escrita

Os buffers armazenam dados temporariamente, permitindo que as threads operem de forma independente e isso evita bloqueios diretos entre as etapas.

PROBLEMAS DE CONCORRÊNCIA

Ao utilizar múltiplas threads com dados compartilhados, **surgem problemas** que precisam ser controlados:

Condição de corrida: Em que as threads acessam o buffer simultaneamente, podendo causar inconsistência nos dados;

Buffer vazio: Em que uma thread tenta consumir dados que ainda não foram produzidos;

Buffer cheio: Em que uma thread tenta inserir dados sem espaço disponível;

Deadlock: Em que threads podem ficar bloqueadas esperando indefinidamente.

SOLUÇÕES IMPLEMENTADAS

MUTEX

O Mutex garante exclusão mútua, permitindo que apenas uma thread acesse o buffer por vez.

```
pthread_mutex_lock(&b->mutex);

b->dados[b->fim] = valor;
b->fim = (b->fim + 1) %
capacidade;
b->quantidade++;

pthread_mutex_unlock(&b->mutex);
```

VARIÁVEIS DE CONDIÇÃO

Elas fazem as threads esperarem quando necessário:

- se o buffer estiver vazio > espera para consumir;
- se o buffer estiver cheio > espera para produzir.

```
while (b->quantidade == 0) {
pthread_cond_wait(&b-
>nao_vazio, &b->mutex);
}
```

FUNCIONAMENTO DO CÓDIGO

```
for (int i = 1; i <= total_itens;
i++) {
    inserir(&b1, i);
}
```

As três threads operam de forma **concorrente.**

Thread de leitura: Gera dados sequencialmente e insere no buffer 1.

```
while (remover(&b1, &valor)) {
    valor = valor * 2;
    inserir(&b2, valor);
}
```

Thread de processamento: Remove dados do buffer 1,realiza uma transformação (ex: multiplicação)e insere no buffer 2.

```
while (remover(&b2, &valor)) {
    printf("valor: %d\n",
valor);
}
```

Thread de escrita: Remove dados do buffer 2 e exibe o resultado final.

EXECUÇÃO DO SISTEMA

```
leitura: 1
leitura: 2
leitura: 3
leitura: 4
leitura: 5
leitura: 6
leitura: 7
processamento: 2
processamento: 4
processamento: 6
processamento: 8
leitura: 8
leitura: 9
leitura: 10
escrita: 2
escrita: 4
escrita: 6
escrita: 8
processamento: 10
processamento: 12
processamento: 14
processamento: 16
processamento: 18
processamento: 20
escrita: 10
escrita: 12
escrita: 14
escrita: 16
escrita: 18
escrita: 20
```

A execução do sistema é não determinística.

A ordem das operações pode variar devido ao escalonamento das threads pelo sistema operacional.

Mesmo assim, todos os dados são processados, não há perda ou duplicação e o fluxo do pipeline é mantido.

OBRIGADA